



# Advanced GPU Programming



Prabhsharan Singh  
MHPC 25-26

Supervisor: Alessandro Gabbana

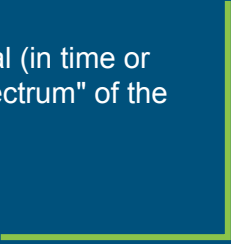




# 1-D Fast Fourier Transform

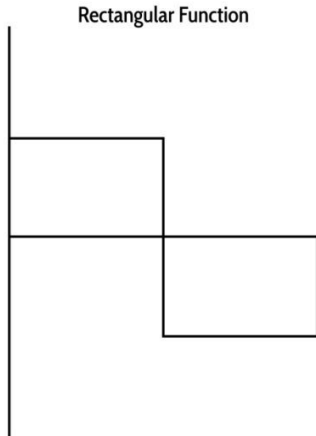


The **Fourier Transform (FT)** is a mathematical operation that takes a signal (in time or space) and decomposes it into its constituent frequencies, revealing the "spectrum" of the data.



# Fourier Transform

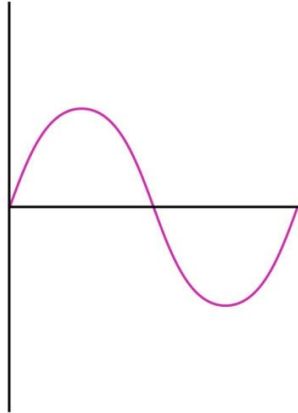
Time Domain



=

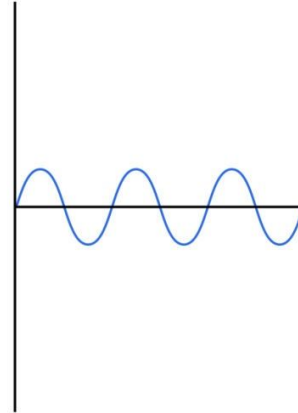
Frequency Domain

1st Wave



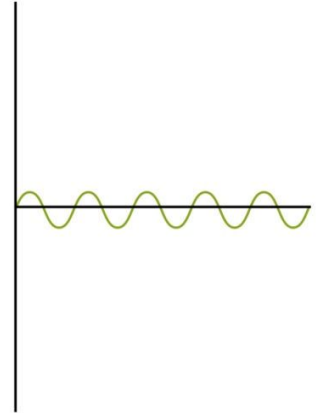
+

2nd Wave

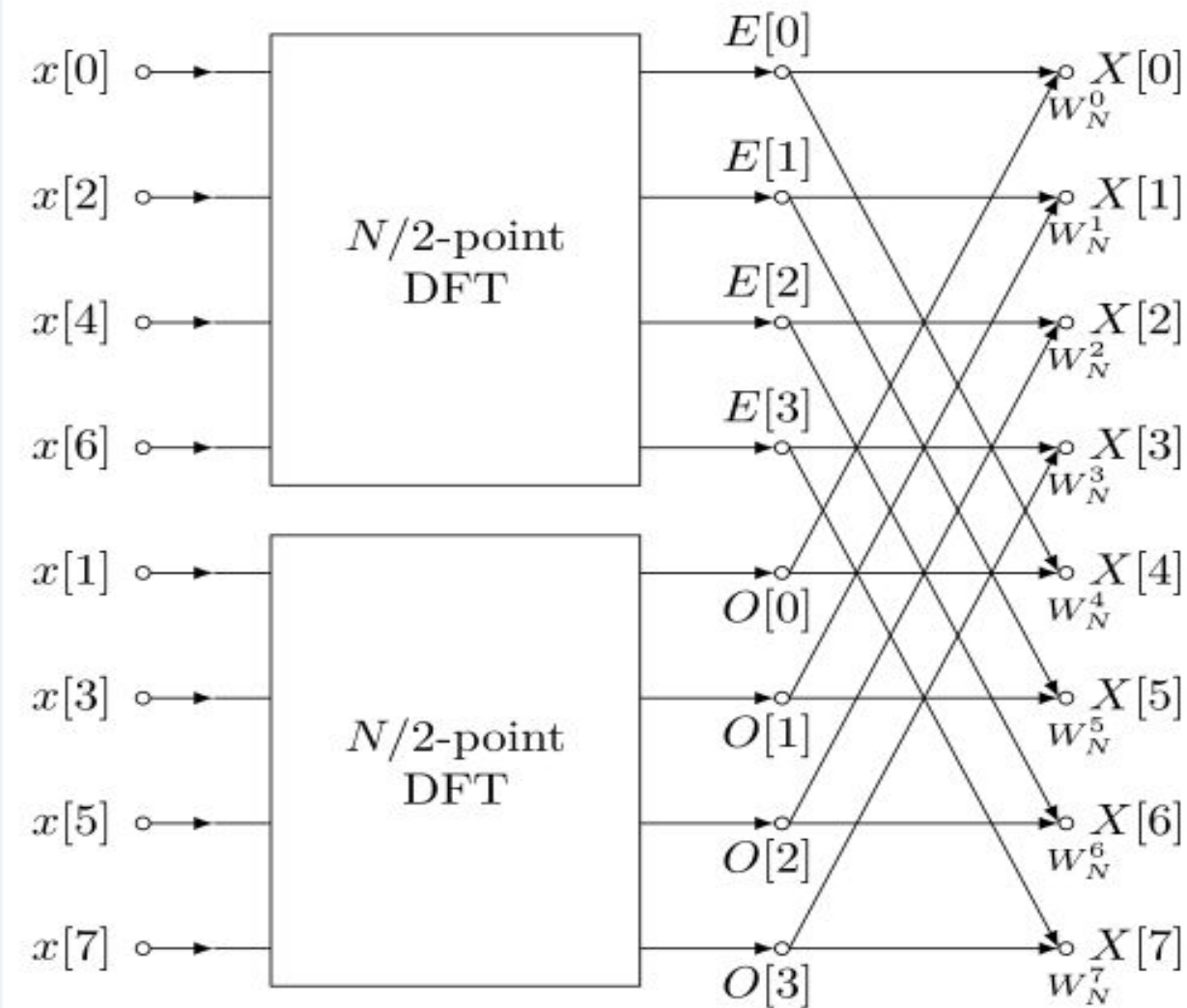


+

3rd Wave



## FAST FOURIER TRANSFORM



DFT  $\rightarrow O(N^2)$

FFT  $\rightarrow O(n \log(n))$

Recursively split the signal into even and odd indexed samples.

In the same loop iteration, we can calculate 4 samples, 2 each for odd and even.

Index of samples  
(n)

| Decimal | Binary |
|---------|--------|
| 0       | 000    |
| 1       | 001    |
| 2       | 010    |
| 3       | 011    |
| 4       | 100    |
| 5       | 101    |
| 6       | 110    |
| 7       | 111    |

Memory

|      |
|------|
| x(0) |
| x(1) |
| x(2) |
| x(3) |
| x(4) |
| x(5) |
| x(6) |
| x(7) |

(Normal Order)

Memory

|      |
|------|
| x(0) |
| x(4) |
| x(2) |
| x(6) |
| x(1) |
| x(5) |
| x(3) |
| x(7) |

(Bit-reversed Order)

Index of samples  
(n)

| Decimal | Binary |
|---------|--------|
| 0       | 000    |
| 4       | 100    |
| 2       | 010    |
| 6       | 110    |
| 1       | 001    |
| 5       | 101    |
| 3       | 011    |
| 7       | 111    |

The cooley-tukey algorithm, recursively divides the signal into even and odd halves, the data naturally ends up in a scrambled order at the bottom of the recursion tree.

By pre-sorting the data into bit-reversed order, the FFT can process the "butterfly" combinations iteratively from the bottom up. This eliminates the need for auxiliary memory arrays, significantly optimizing memory usage and cache performance.

## FAST FOURIER TRANSFORM

### Sequential CPU Implementation

```
void fft_1d(Complex *data, int N) {
    int log2n = 0;
    // Calculate the number of stages (log2(N))
    for (int temp = N; temp > 1; temp >>= 1)
        log2n++;

    /* Bit-reversal permutation */
    for (unsigned int i = 0; i < (unsigned int)N; i++) {
        unsigned int j = bit_reverse(i, log2n); // get the bit reversed index
        if (i < j) { //
            Complex tmp = data[i];
            data[i] = data[j];
            data[j] = tmp;
        }
    }

    /* Cooley-Tukey */
    for (int s = 1; s <= log2n; s++) { // Iterate through each of the log2(N) stages
        int m = 1 << s; // Current sub-problem size (dual node distance)
        int m2 = m >> 1; // Half-size of the current block (offset for butterflies)

        // get the base twiddle factor
        Complex w = c_exp_i(-2.0 * M_PI / m);

        for (int k = 0; k < N; k += m) { // Iterate through each DFT group in the current stage
            Complex wn = c_make(1.0, 0.0); // get unit twiddle factor

            for (int j = 0; j < m2; j++) { // Perform the butterfly operations within the group

                Complex t = c_mul(wn, data[k + j + m2]); // Compute the "Odd" branch multiplied by twiddle factor
                Complex u = data[k + j]; // Capture the "Even" branch value

                data[k + j] = c_add(u, t); // top half
                data[k + j + m2] = c_sub(u, t); // bottom half

                wn = c_mul(wn, w); // Update twiddle factor to the next power:
            }
        }
    }
}
```

```
Complex c_exp_i(double theta) {
    return c_make(cos(theta),
                  sin(theta));
}
```

$$X_k = \underbrace{\sum_{m=0}^{N/2-1} x_{2m} e^{-i \frac{2\pi}{N} km}}_{\text{DFT of Evens } (E_k)} + e^{-i \frac{2\pi}{N} k} \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} e^{-i \frac{2\pi}{N} km}}_{\text{DFT of Odds } (O_k)}$$

# FAST FOURIER TRANSFORM

## CUDA Accelerated V1

```
/**
 * Naive implementation of fft_1d
 */
float fft_1d_gpu_v1(Complex *h_data, int N) {
    Complex *d_data; // device pointer
    size_t size = N * sizeof(Complex); // problem size

    // cuda config
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
    int total_threads = N / 2; // We need N/2 butterflies per stage
    int grid_size = (total_threads + threadsPerBlock - 1) / threadsPerBlock;

    int log2n = 0;
    for (int temp = N; temp > 1; temp >= 1) log2n++; // total FFT stages

    cudaMalloc((void **)&d_data, size);
    cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);
    cudaEventRecord(start); // start timer

    bit_reverse_kernel<<<blocks, threadsPerBlock>>>(d_data, N, log2n);
    cudaDeviceSynchronize();

    for (int s = 1; s <= log2n; s++) {
        int m = 1 << s; // Current sub-problem size (2, 4, 8...)
        int m2 = m >> 1; // Stride/Offset for the butterfly (1, 2, 4...)
        fft_butterfly_kernel<<<grid_size, threadsPerBlock>>>(d_data, N, m, m2);
    }

    cudaEventRecord(stop); // stop timer
    cudaEventSynchronize(stop);

    // time evaluation
    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);
    printf("GPU V1 Time (Global): %.4f ms\n", ms);

    // memory cleanup
    cudaMemcpy(h_data, d_data, size, cudaMemcpyDeviceToHost);
    cudaFree(d_data);
    return ms;
}
```

```
__global__ void bit_reverse_kernel(Complex *data, int N, int log2n) {
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        unsigned int n = i, j = 0;
        for (int b = 0; b < log2n; b++) { j = (j << 1) | (n & 1); n >>= 1; }
        if (i < j) { Complex tmp = data[i]; data[i] = data[j]; data[j] = tmp; }
    }
}

// V1 Kernel
__global__ void fft_butterfly_kernel(Complex *data, int N, int m, int m2) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N / 2) {
        int k = tid / m2;
        int j = tid % m2;
        int idx1 = k * m + j;
        int idx2 = idx1 + m2;
        Complex w = dev_c_exp_i(-2.0 * M_PI * j / m);
        Complex u = data[idx1];
        Complex t = dev_c_mul(w, data[idx2]);
        data[idx1] = dev_c_add(u, t);
        data[idx2] = dev_c_sub(u, t);
    }
}
```

## FAST FOURIER TRANSFORM

### CUDA Accelerated V2, Using Shared Memory

```
/**
 * Shared memory implementation of fft_1d
 */
float fft_1d_gpu_v2(Complex *h_data, int N) {
    Complex *d_data;
    size_t size = N * sizeof(Complex);
    int log2n = 0;
    for (int temp = N; temp > 1; temp >= 1) log2n++;

    cudaMalloc((void **)&d_data, size);
    cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);
    int threads = N / 2;
    size_t shared_mem_size = N * sizeof(Complex);

    // cuda config
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEventRecord(start);
    bit_reverse_kernel<<<blocks, threadsPerBlock>>>(d_data, N, log2n);
    fft_shared_kernel<<<1, threads, shared_mem_size>>>(d_data, N, log2n); // shared memory, hence 1 huge block
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);
    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);
    printf("GPU V2 Time (Shared):   %.4f ms\n", ms);
    cudaMemcpy(h_data, d_data, size, cudaMemcpyDeviceToHost);
    cudaFree(d_data);
    return ms;
}
```



```

// V2 Kernel (Shared, Unpadded)
__global__ void fft_shared_kernel(Complex *data, int N, int log2n) {
    extern __shared__ Complex s_data[]; // dynamic allocation of shared memory based on kernel args
    int tid = threadIdx.x; // local thread id

    int idx1 = tid; // element 1 the thread will handle
    int idx2 = tid + blockDim.x; // element 2 the thread will handle

    // load from global to shared memory
    if (idx1 < N) s_data[idx1] = data[idx1];
    if (idx2 < N) s_data[idx2] = data[idx2];
    __syncthreads();

    // FFT
    for (int s = 1; s <= log2n; s++) {
        int m = 1 << s; // sub-problem size
        int m2 = m >> 1; // stride
        int k = tid / m2; // butterfly group idx
        int j = tid % m2; // butterfly item idx

        int s_idx1 = k * m + j; // top part, in shared mem
        int s_idx2 = s_idx1 + m2; // bottom part, in shared mem

        // Butterfly Math (u = even, t = odd * twiddle)
        Complex w = dev_c_exp_i(-2.0 * M_PI * (double)j / (double)m);
        Complex u = s_data[s_idx1];
        Complex t = dev_c_mul(w, s_data[s_idx2]);

        // write back to shared mem
        s_data[s_idx1] = dev_c_add(u, t);
        s_data[s_idx2] = dev_c_sub(u, t);

        __syncthreads();
    }

    // write back to global mem
    if (idx1 < N) data[idx1] = s_data[idx1];
    if (idx2 < N) data[idx2] = s_data[idx2];
}

```

## FAST FOURIER TRANSFORM

### CUDA Accelerated V3, Using Shared Memory + Padding

```
/**
 * Shared memory + fixing bank conflicts
 */
float fft_1d_gpu_v3(Complex *h_data, int N) {
    Complex *d_data;
    size_t size = N * sizeof(Complex);
    int log2n = 0;
    for (int temp = N; temp > 1; temp >= 1) log2n++;
    cudaMalloc((void **)&d_data, size);
    cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);

    // cuda config
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
    int threads = N / 2;
    size_t padded_elements = N + (N >> 5); // 32 threads in each warp
    size_t shared_mem_size = padded_elements * sizeof(Complex);

    cudaEventRecord(start);
    bit_reverse_kernel<<<blocks, threadsPerBlock>>>(d_data, N, log2n);
    fft_padded_kernel<<<1, threads, shared_mem_size>>>(d_data, N, log2n);
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);

    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);
    printf("GPU V3 Time (Padded):  %.4f ms\n", ms);
    cudaMemcpy(h_data, d_data, size, cudaMemcpyDeviceToHost);
    cudaFree(d_data);
    return ms;
}
```

```

// V3 Kernel (Shared + Padding Optimization)
__global__ void fft_padded_kernel(Complex *data, int N, int log2n) {
    extern __shared__ Complex s_data[]; // dynamically allocated shared memory with padding

    int tid = threadIdx.x;
    int idx1 = tid;
    int idx2 = tid + blockDim.x;

    // use padded index
    // load data from global to shared memory
    // with padding of one element
    if (idx1 < N) s_data[PAD(idx1)] = data[idx1];
    if (idx2 < N) s_data[PAD(idx2)] = data[idx2];

    __syncthreads();

    // FFT
    for (int s = 1; s <= log2n; s++) {
        int m = 1 << s;
        int m2 = m >> 1;
        int k = tid / m2;
        int j = tid % m2;

        int s_idx1 = k * m + j;
        int s_idx2 = s_idx1 + m2;

        Complex w = dev_c_exp_i(-2.0 * M_PI * (double)j / (double)m);

        // use padded memory location for no bank conflict
        Complex u = s_data[PAD(s_idx1)];
        Complex t = dev_c_mul(w, s_data[PAD(s_idx2)]);

        // write the data in correct place in shared memory
        s_data[PAD(s_idx1)] = dev_c_add(u, t);
        s_data[PAD(s_idx2)] = dev_c_sub(u, t);

        __syncthreads();
    }

    // write data back from padded shared memory into global memory
    if (idx1 < N) data[idx1] = s_data[PAD(idx1)];
    if (idx2 < N) data[idx2] = s_data[PAD(idx2)];
}

```

## FAST FOURIER TRANSFORM

OpenACC Accelerated version, without  
Shared Memory

```
float fft_1d_gpu_acc_v1(Complex *data, int N) {
    int log2n = 0;
    for (int temp = N; temp > 1; temp >= 1) log2n++;
    clock_t start = clock();
    #pragma acc data copy(data[0:N])
    {
        #pragma acc parallel loop independent
        for (unsigned int i = 0; i < N; i++) {
            unsigned int n = i, j = 0;
            for (int b = 0; b < log2n; b++) { j = (j << 1) | (n & 1); n >>= 1; }
            if (i < j) { Complex tmp = data[i]; data[i] = data[j]; data[j] = tmp; }
        }
        for (int s = 1; s <= log2n; s++) {
            int m = 1 << s, m2 = m >> 1;
            #pragma acc parallel loop independent gang
            for (int k = 0; k < N; k += m) {
                #pragma acc loop independent vector
                for (int j = 0; j < m2; j++) {
                    Complex w = c_exp_i(-2.0 * M_PI * j / m);
                    Complex u = data[k + j];
                    Complex t = c_mul(w, data[k + j + m2]);
                    data[k + j] = c_add(u, t);
                    data[k + j + m2] = c_sub(u, t);
                }
            }
        }
    }
    clock_t stop = clock();
    return (float)(stop - start) * 1000.0 / CLOCKS_PER_SEC;
}
```

## FAST FOURIER TRANSFORM

### Using cuFFT Library

```
float fft_1d_cufft(Complex *h_data, int N) {
    cufftHandle plan;
    cufftDoubleComplex *d_data;
    size_t size = N * sizeof(Complex);

    // 1. Allocate & Copy to GPU
    cudaMalloc((void**)&d_data, size);
    cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);

    // 2. Create Plan & Execute
    if (cufftPlan1d(&plan, N, CUFFT_Z2Z, 1) != CUFFT_SUCCESS) return 0.0f;

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);

    cudaEventRecord(start);
    cufftExecZ2Z(plan, d_data, d_data, CUFFT_FORWARD);
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);
    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);

    // 3. Copy Back & Cleanup
    cudaMemcpy(h_data, d_data, size, cudaMemcpyDeviceToHost);
    cufftDestroy(plan);
    cudaFree(d_data);

    return ms;
}
```

```

#!/bin/bash
#SBATCH --job-name=fft_acc_run
#SBATCH --nodes=1
#SBATCH --gres=gpu:v100:1
#SBATCH --partition=skyvolta
#SBATCH --time=00:05:00
#SBATCH --output=fft_acc-%j.out
#SBATCH --error=fft_acc-%j.err
#SBATCH --exclusive

# 1. Clean up old files
rm -rf *.x *.txt

# 2. Load Environments
module load cuda/11.8
module load nvhpc/24.7

# Define Problem Size
N=$((2**10))

echo "====="
echo "Compiling OpenACC Version (using nvc++)..."
echo "====="
# Added -cudalib=cufft so nvc++ links the CUDA math library
nvc++ -acc -cuda -Minfo=all -gpu=cc70 -O3 fft_acc_v2.cu -o fft_acc.x -cudalib=cufft

echo "====="
echo "Running Benchmarks (N=$N)"
echo "====="

srun ./fft_acc.x $N

echo "====="
echo "Done. Generated files:"
ls -l *.txt
~

```

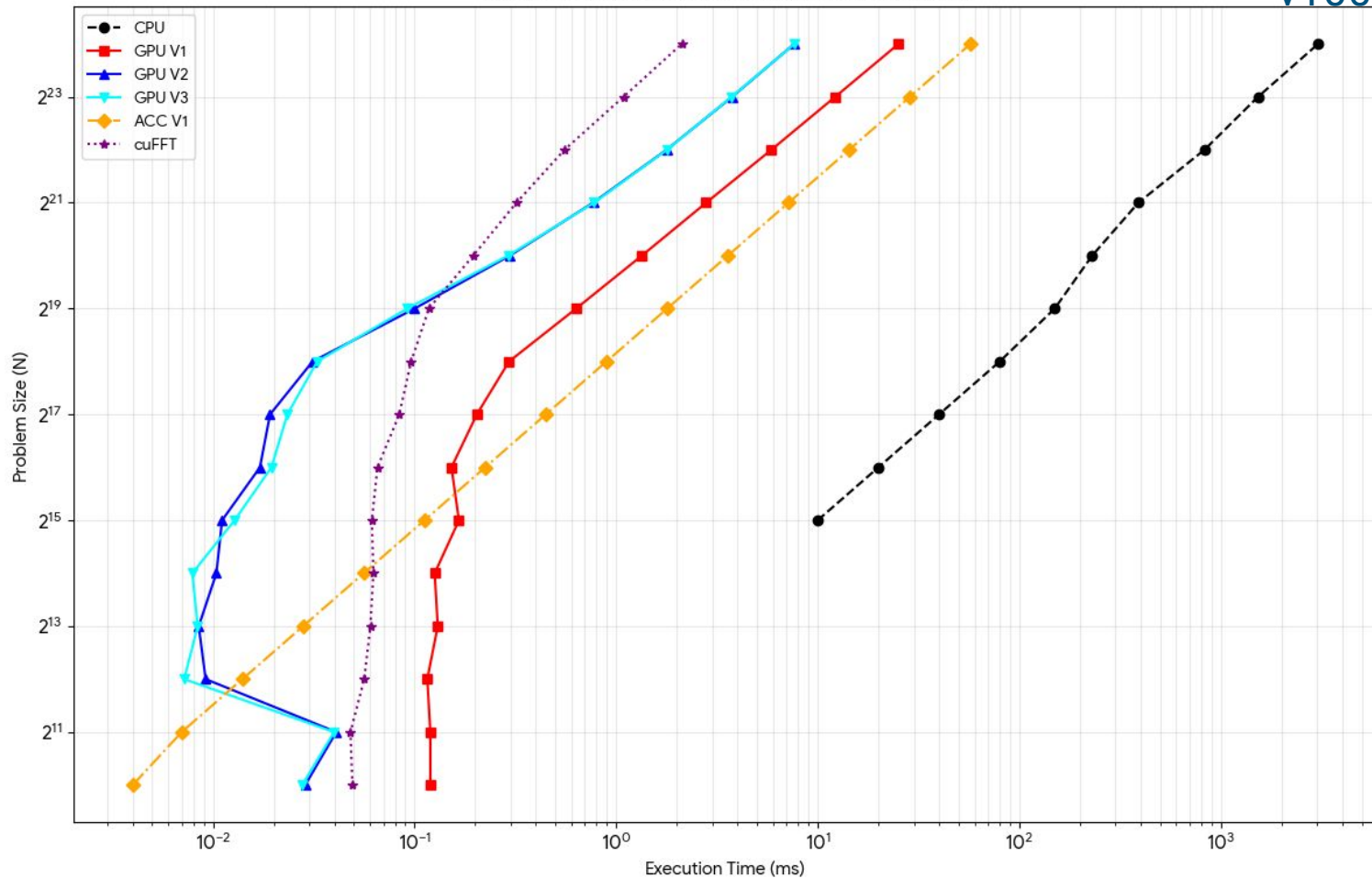
## Slurm Script

## Using COKA Cluster

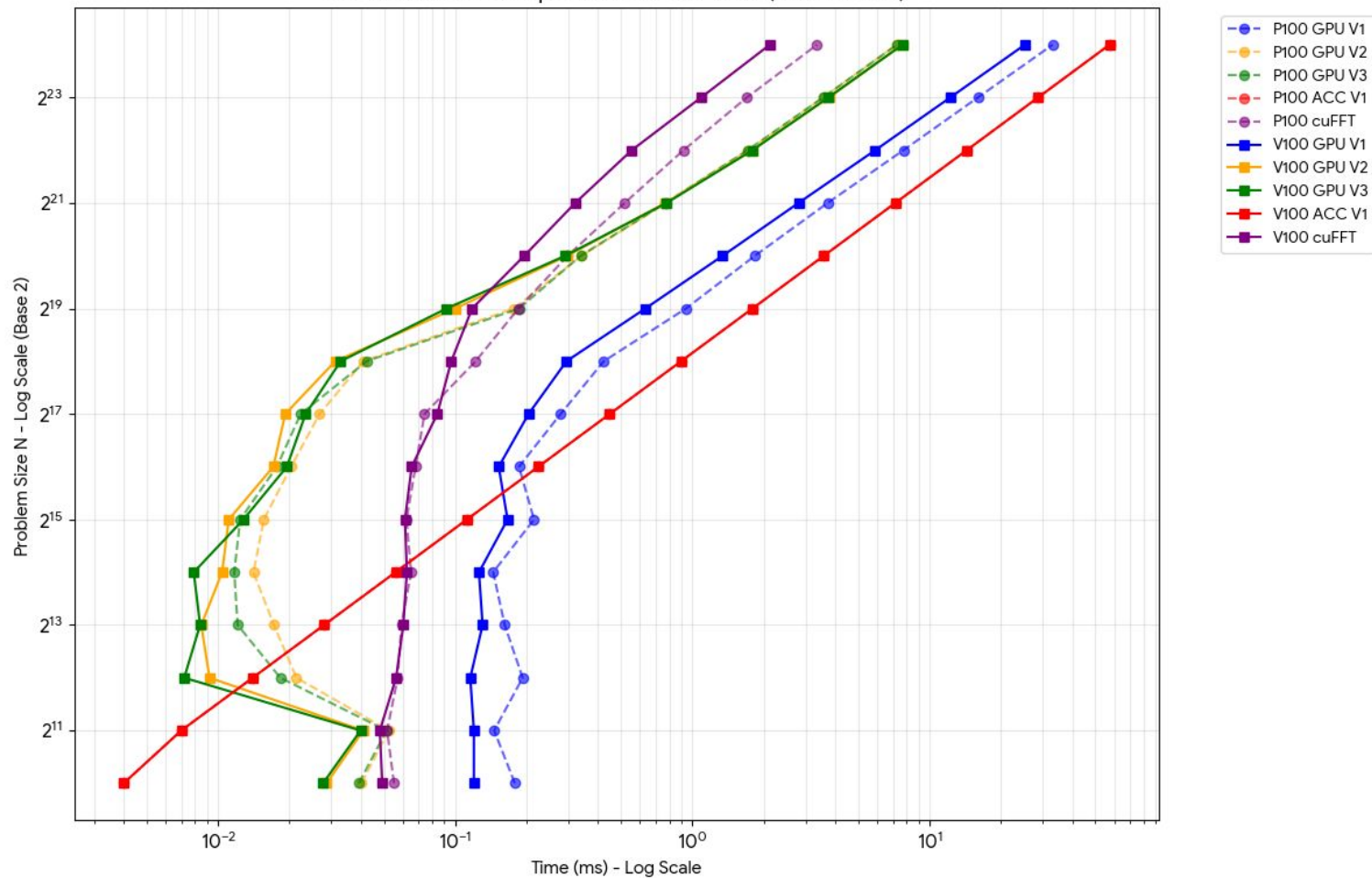


FFT Benchmark: Execution Time vs Problem Size

v100



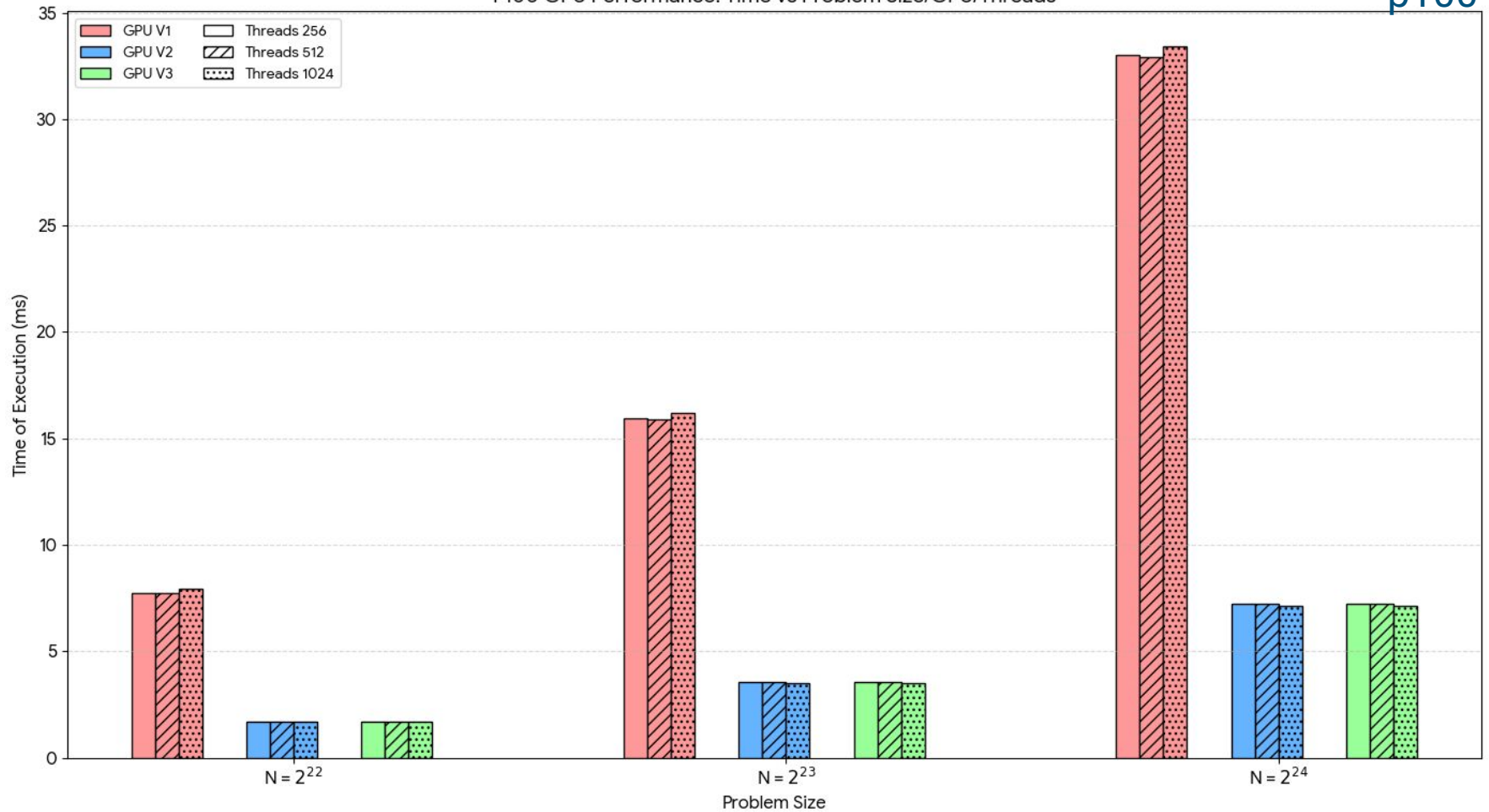
FFT Benchmark Comparison: P100 vs V100 (Time vs Size)





P100 GPU Performance: Time vs Problem Size/GPU/Threads

p100





# Thanks.



Q&A

